Does the enforcement of legal play through logit masking and environment constraints accelerate agent convergence?

by

Francois Qian
URN: 6702759/7

A dissertation submitted in partial fulfilment of the
requirements for the award of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

May 2026

Department of Computer Science
University of Surrey
Guildford GU2 7XH

Supervised by: Nishanth Sastry

I declare that this dissertation is my own work and that the work of others is acknowledged and indicated by explicit references.

Francois Qian
May 2026

© Copyright Francois Qian, May 2026

Abstract

1. The convergence of self-play in a pseudo-legal environment toward strict legal play.
2. The impact of logit masking on learning dynamics, specifically regarding the "grokking" of physical board mechanics.

In the development of neural agents for combinatorial games, a fundamental tension exists between explicit policy pruning and emergent rule internalization. This thesis investigates two primary paradigms: **Logit Masking** and **Punishment Propagation**.

- **Hypothesis:** While logit masking provides superior convergence speed, unmasked training (Punishment Propagation) forces the network to "grok" the underlying physical mechanics of the environment, potentially yielding more robust latent representations.

Acknowledgements

I would like to thank my supervisor, Nishanth Sastry, for overseeing the project.

Contents

1 Introduction **14**
1.1 Project Background 14
1.2 Project Description 14
1.3 Aims and Objectives 15
1.4 Thesis Structure 15

2 Literature Review **16**
2.1 Chess as an MDP 16
2.2 Chess Played by Machines 16
2.2.1 AlphaZero and MuZero 16
2.2.2 Transformers in Chess 17
2.3 Invalid Action Masking 17
2.4 Learning Dynamics 17
2.5 Summary and Research Gap 18

3 System Design and Methodology **19**
3.1 System Architecture Overview 19
3.2 Environment Formulation and State Representation 19
3.2.1 Dual Ruleset MDPs 19
3.2.2 State Canonicalization and Tensor Formulation 19
3.2.3 Network Topology 20
3.2.4 Output Heads 20
3.2.5 Bipartite Monte Carlo Tree Search (MCTS) 20
3.2.6 Selection and PUCT Formulation 20
3.2.7 Exploration Mechanisms 21
3.2.8 Simulation Integrity and Masking 21
3.3 Reinforcement Learning Pipeline 21
3.3.1 Self-Play and Replay Buffer 21
3.4 Optimisation and Loss Function 21
3.5 Experimental Design and Evaluation Metrics 22
3.5.1 The Configuration Matrix 22
3.5.2 Quantitative Metrics 22

4 Implementation Details **23**
4.1 Software Stack and Overview 23

4.2 Chess Engine ... 24
4.2.1 Bitboards and Move Generation ... 24
4.2.2 Zobrist hashing: ... 25
4.3 Bipartite Monte Carlo Tree Search ... 25
4.3.1 Arena Allocators ... 25
4.3.2 Node Expansion, Promotions and Terminal Handling ... 26
4.3.3 Edge Selection and Value Propagation ... 27
4.3.4 Mask Integration and Prior Renormalisation ... 28
4.4 Neural Network Architecture ... 28
4.4.1 Tensor Formulation and Canonicalisation: ... 28
4.4.2 Transformer Encoder and Dynamic Masking ... 28
4.5 Training Pipeline ... 28
4.5.1 Batch Expansion and Synchronisation ... 29
4.5.2 Self-Annealing Loss Function ... 29
4.6 Performance and Correctness ... 30

5 Results and Evaluation ... 31
5.1 Experimental Setup ... 31
5.2 Ruleset Convergence Analysis ... 31
5.3 Masking and Grokking Analysis ... 31

6 Conclusion ... 32
6.1 Project Evaluation ... 32
6.2 Limitations and Future Work ... 32

7 Statement of Ethics ... 33

Bibliography ... 34

List of Figures

List of Tables

Glossary

$Q(s, a)$	The total reward accumulated from state
$C(s)$	CPUCT
S_i	Upper confidence bound
$\overline{x_i}$	The Mean Value of the current node
c	The Exploitation parameter
N	The current node's Visit Count
n_i	The Visit Count of the child

Abbreviations

MCTS	Monte Carlo Tree Search
UCB1	Upper Confidence Bound 1
PUCT	Predictor + Upper Confidence Bound applied to Trees
CPUCT	Exploration Predictor Constant

Chapter 1

Introduction

1.1 Project Background

Modern machine learning often relies on scaling data and compute over opaque architectures. However, these models are frequently decoupled from environment transition dynamics via **Logit Masking**—a heuristic that artificially zeroes out the probability of illegal actions prior to softmax activation. While computationally efficient, this deprives the network of negative feedback for invalid predictions, bypassing the model's need to internally represent the mechanics of play. Furthermore, standard reinforcement learning (RL) agents are prevented from making invalid state transitions by hardcoded environmental constraints. This precludes the observation of how internal action space pruning influences learning before strategic evaluation develops. Chess provides an optimal substrate for this investigation: it presents highly ambiguous intermediate states bound by objective ground truths and a mathematically perfect, yet computationally intractable, solution space.

1.2 Project Description

Existing literature rarely assesses the impact of these environmental guardrails on the learning dynamics of transformer-based agents. This project investigates two orthogonal axes of environmental constraint: Action Space Constraints and Logit Masking.

To facilitate explicit measurement of model interpretability, a custom two-pass autoregressive transformer encoder and a bipartite Monte Carlo Tree Search (MCTS) are introduced. This architecture decouples piece selection from destination selection, providing a granular window into the model's spatial reasoning.

The experimental configurations are defined across two axes:

1. **Axis 1: Rule Set Convergence (Action Space Constraints)**
   - **Control:** Training on a strict legal ruleset.
   - **Test:** Training on a pseudo-legal ruleset where the terminal state is the simpler, denser king capture $\left(K_{\text{cap}}\right)$ over the highly sparse, complex $(A_l = \emptyset)$. This variant of Chess fundamentally alters the MDP reward function to $R(s, a)$ yielding a potentially smoother reward landscape.

- **Hypothesis:** The network first bootstraps learning the rules and goals of the environment. Ideal play in the pseudo-legal space A_P may converge to standard Chess where concepts such as pins and checkmates emerge as survival heuristics to prevent or secure king capture. Conversely, the removal of environmental guardrails may degrade MCTS value estimation, preventing policy bootstrapping.

2. **Axis 2: Logit Masking**
   - **Control:** Masked logits (illegal moves are mathematically eliminated prior to selection).
   - **Test:** Unmasked logits (illegal moves are permitted by the network but penalized by the environment/loss function).
   - **Hypothesis:** Unmasked training will incur an initial sample-efficiency penalty by introducing the dual-optimisation problem of learning game mechanics alongside strategy, but may lead to more robust internal representations reflecting potentially stronger play long term.

1.3 Aims and Objectives

The primary aim of this project is to train multiple transformer-based agents via self-play reinforcement learning across varying rule sets and action spaces, quantitatively evaluating their performance, learning trajectories, and emergent behaviors.

Specific objectives include:

- Implement a highly optimized, dual-ruleset (legal and pseudo-legal) Chess library to facilitate high-throughput RL self-play.
- Architect and implement a two-pass autoregressive transformer and bipartite MCTS to explicitly model piece-then-square action selection.
- Train four distinct model configurations via self-play: (Legal, Masked), (Legal, Unmasked), (Pseudo-Legal, Masked), and (Pseudo-Legal, Unmasked).
- Quantitatively evaluate convergence rates, illegal move proposal frequencies, and the emergence of rule-abiding play across the unconstrained configurations.
- Correlate the emergence of legal play with the attention distributions of the two-pass encoder to quantify spatial reasoning.

1.4 Thesis Structure

The remainder of this dissertation is structured as follows:

- **Chapter 2** reviews the literature surrounding reinforcement learning, transformer architectures, and action masking.
- **Chapter 3** details the system design, including the bipartite MCTS and two-pass encoder architecture.
- **Chapter 4** outlines the implementation details of the Rust engine and training pipeline.
- **Chapter 5** presents the results and evaluation of the four experimental configurations.
- **Chapter 6** concludes the project and discusses avenues for future work.

Chapter 2

Literature Review

2.1 Chess as an MDP

Chess is a deterministic, zero-sum, perfect-information game. It is formally modeled as a Markov Decision Process (MDP) defined by a tuple (S, A, T, R), where S is the state space ($|S| \approx 10^{\{40\}}$), A is the action space, $T : S \times A \to S$ is the deterministic transition function, and $R : S \times A \to \{-1, 0, 1\}$ is the reward function (Shannon, 1950).

Standard RL implementations enforce a legal action space $A_{L(s)} \subset A$ where A_L contains only moves that comply with the rules of chess (e.g., preventing self-check) (World Chess Federation (FIDE), 2023). This project introduces a relaxed pseudo-legal action space $A_{P(s)}$ where $A_{L(s)} \subset A_{P(s)} \subset A$, permitting all geometrically valid piece movements and shifting the terminal condition from the abstract checkmate to the explicit king capture $\left(K_{\text{cap}}\right)$.

Typical agents produce a single policy distribution representing every possible piece movement on all 64 squares ($|A| = 4672$). Factoring the joint probability $P(a \mid s)$ into $P(\text{from} \mid s) \times P(\text{to} \mid s, \text{from})$ reduces output layer parameters significantly (e.g., from d_model $\times$ 4672 to d_model $\times$ 64).

For any state s, an optimal policy π^* trained in A_P must implicitly learn to prune the action space, such that:

$$\pi^*(a \mid s) \to 0 \quad \text{for all } a \in \left(A_{P(s)} \setminus A_{L(s)}\right)$$

In this paradigm, concepts such as pins and checkmates are not hardcoded environmental constraints, but emergent survival heuristics within the value landscape.

2.2 Chess Played by Machines

2.2.1 AlphaZero and MuZero

The paradigm of self-play RL in chess was defined by AlphaZero (Silver *et al.*, 2017), which combined Monte Carlo Tree Search (MCTS) with deep convolutional neural networks (CNNs) for policy and value evaluation. However, AlphaZero relies entirely on a hardcoded environment to generate $A_{L(s)}$, bypassing the need for the network to internalize the rules of the game.

Subsequent architectures, such as MuZero (Schrittwieser *et al.*, 2020), removed the need for a known transition dynamics model by learning a latent environment simulator. While MuZero demonstrates the ability to adhere to environment constraints implicitly within its hidden state transitions, its policy head still simulates within the legal action space during MCTS rollouts. The literature lacks insight into how the removal of environmental guardrails or action masking impacts the sample efficiency and representational robustness of the underlying network.

2.2.2 Transformers in Chess

Recent advancements demonstrate the efficacy of Transformer architectures in chess, shifting away from the spatial inductive biases of CNNs. (Monroe and Chalmers, 2024) demonstrated that Transformers can achieve Grandmaster-level performance without explicit search, relying entirely on attention mechanisms to evaluate board states. Furthermore, (Jenner *et al.*, 2024) provided evidence of learned look-ahead within chess-playing neural networks, showing that attention heads naturally encode future board states.

Standard chess engines flatten the action space into a single discrete distribution. Decoupling piece selection $P(\text{from} \mid s)$ and destination selection $P(\text{to} \mid s, \text{from})$ forces attention mechanisms to explicitly map geometric constraints. This factored approach provides an observable window into distinct failure modes: attention failure (identifying movable pieces) and kinematic failure (how pieces move).

2.3 Invalid Action Masking

In policy gradient algorithms, it is standard practice to prevent the selection of illegal actions via Invalid Action Masking (Logit Masking). Masking applies a transformation to the policy logits z prior to the softmax activation:

$$P(a_i \mid s) = \frac{\exp(z_i + M_i)}{\sum_j \exp\big(z_j + M_j\big)}$$

where $M_i = -\infty$ if $a_i \notin A_{L(s)}$, and 0 otherwise.

While employed to accelerate convergence, (Huang and Ontañón, 2020) demonstrated that invalid action masking significantly improves sample efficiency by collapsing the exploration space. However, masking prevents punishment from invalid actions from propagating back through the network. This is critical for MCTS algorithms, where simulations into invalid state transitions degrade search efficiency and poison the policy head.

2.4 Learning Dynamics

The requirement for an unmasked network to learn the rules of chess from scratch introduces complex learning dynamics. (Power *et al.*, 2022) identified the phenomenon of "grokking," where neural networks trained on small algorithmic datasets exhibit delayed generalisation long after overfitting the training data. The phenomenon arose from the result of weight decay pressure, showcasing the importance of weight regularisation. Chess is an algorithmic dataset embedded within a strategic MDP, showcasing the potential for analogous phase transitions.

(Bengio *et al.*, 2009) formalized curriculum learning, finding that machine learning models converge more effectively if training data is organized in a sequence of increasing complexity. The authors demonstrate this strategy effectively functions as a regularisation method, guiding the model to superior local minima often inaccessible through data shuffling. Theoretically, a pseudo-legal environment serves as an implicit curriculum, where an agent first receives rewards through piece kinematics and king capture before developing legal survival heuristics.

(Sharma *et al.*, 2017) introduced a framework that exploits the compositional nature of discrete action spaces by factoring them into independent basis-like components. This enables cross-action learning, allowing the agent to generalize actions that share common factors.

2.5 Summary and Research Gap

While the efficacy of action masking is well-documented in general RL (Huang and Ontañón, 2020), and Transformers have proven capable of modeling chess (Jenner *et al.*, 2024), the intersection of these domains remains unexplored. Existing literature does not isolate the impact of environmental guardrails on the representational quality of Transformer-based MCTS agents.

In the context of unmasked, pseudo-legal chess training, the network faces a dual-optimisation problem: it must learn the physical constraints of the board (the rules) and the strategic evaluation of states (the game). It is hypothesized that the network will exhibit a grokking-like phase transition: an initial period of high illegal-move proposals (random walk), followed by a sharp convergence toward $A_{L(s)}$ as physical constraints are internalized, preceding the development of high-level strategy.

This project addresses this gap by training four distinct configurations across two orthogonal axes: Action Space Constraints (Legal vs. Pseudo-Legal) and Logit Masking (Masked vs. Unmasked). By utilizing a factored two-pass autoregressive encoder, this research provides a novel, quantitative analysis of how rule internalization and action space pruning manifest within the attention layers of a chess-playing agent.

Chapter 3

System Design and Methodology

3.1 System Architecture Overview

The system is designed as a closed-loop reinforcement learning pipeline, decoupling the environment transition dynamics from the neural network's internal representations. The architecture consists of three primary components: a custom deterministic environment simulator supporting dual rulesets, a two-pass autoregressive Transformer encoder, and a bipartite Monte Carlo Tree Search (MCTS) algorithm. The system operates via self-play, generating trajectories that are aggregated into a global replay buffer to optimize the network via gradient descent.

3.2 Environment Formulation and State Representation

3.2.1 Dual Ruleset MDPs

1. **Legal Environment ($\boldsymbol{E_l}$)**: Adheres to standard FIDE rules. The action space ($A_{l(s)}$) is strictly contained to moves that do not leave the king in check. The terminal reward $R(s, a)$ is evaluated upon Checkmate, Stalemate or standard draw conditions.
2. **Pseudo-Legal Environment ($\boldsymbol{E_p}$)**: Relaxes the action space ($A_{p(s)}$), permitting all geometrically valid piece movements regardless of king safety. The terminal conditions is shifted to the explicit king capture (K_{cap}). FIDE draw conditions are maintained to prevent infinite rollouts and support correct transition into legal play.

3.2.2 State Canonicalization and Tensor Formulation

To enforce symmetric learning and halve the state space, all board states are canonicalized to the perspective of the side-to-move. If it is Black's turn, the board and castling rights are geometrically flipped. The state s is mapped to the spatial tensor representation $X \in \mathbb{R}^{64\times 14}$ and a scalar meta-tensor $M \in \mathbb{R}^5$.

- **Spatial Tensor**: Comprises 12 piece bitboards (6 piece types $\times$ 2 colors), 1 en-passant target plane, and 1 selected-square plane (utilized in the second pass of the network).
- **Meta Tensor**: Encodes the 4 castling rights (one-hot) and the half-move clock (scaled by $\frac{1}{100}$).

To explicitly measure spatial reasoning, the system factors the joint probability of a move $P(a|s)$ into an autoregressive sequence: $P(a|s) = P(\text{from}|s) \times P(\text{to}|s, \text{from})$. The network executes two forward passes using shared weights:

1. **Pass 1 (Piece Selection)**: The network evaluates the board state with the selected square plane zeroed out, outputting a policy distribution over the 64 squares to select the origin square (*from*). The value head will evaluate the board state.
2. **Pass 2 (Destination Selection)**: The origin square is encoded into the 14th plane of the spatial tensor. The network re-evaluates the state, outputting a policy distribution over the 64 squares to select the destination (*to*). The value head will evaluate the position given a piece to move, effectively evaluating the viability of the selected piece.

Note on Promotions: Because the factored $64 \to 64$ architecture lacks a third dimension for promotion piece selection, promotions are handled dynamically during MCTS expansion. If a pawn reaches the 8th rank, the destination edge is expanded into four distinct sub-edges (Q, R, B, N), dividing the prior probability equally.

3.2.3 Network Topology

The model uses a Transformer Encoder architecture without spatial inductive biases (e.g., convolutions). The spatial tensor X is linearly projected to d_{model}, and $2D$ learned positional embeddings are added to retain geometric context. The meta-tensor M is linearly projected and concatenated as a pseudo-[CLS] token, resulting in a sequence length of 65.

The model is scaled symmetrically:

- $n_{\text{heads}} = 8$, $n_{\text{layers}} = 8$, $d_{\text{model}} = 8 * 64$, $d_{\text{ff}} = 8 * 4 * 64$

3.2.4 Output Heads

The final latent representation is routed into two distinct heads:

1. **Policy Head**: A linear projection of the 64 spatial tokens to $\mathbb{R}^{64}$, representing unnormalized logits for square seletion.
2. **Value Head**: A linear projection of the [CLS] token to $\mathbb{R}^3$, passed through softmax to predict Win/Draw/Loss probabilities. In Pass 1, this evaluates the raw state; in Pass 2, it evaluates the state conditioned on the highlighted origin square.

3.2.5 Bipartite Monte Carlo Tree Search (MCTS)

To accomodate the two-pass autoregressive policy, a bipartite MCTS is implemented. The search tree alternates between two node types:

- **Selection Nodes (N_{select})**: Represents a board state. Edges represent selection of an origin square.
- **Action Nodes (N_{move})**: Represents a board state plus selected origin square. Edges represent the destination square, transitioning the environment to an N_{select} node.

3.2.6 Selection and PUCT Formulation

During traversals, edges are selected using a modified Predictor + Upper Confidence Bound applied to Trees (PUCT) algorithm. To mitigate draw-seeking behaviour, a contempt factor is baked into the empirical action value $Q(s, a)$.

- $Q(s, a) = W - L - 0.05D$
- $U(s, a) = c_{\text{puct}} P(s, a) \frac{\sqrt{\sum_b N(s,b)}}{1+N(s,a)}$(Exploration)

- $\text{PUCT}(s, a) = Q(s, a) + U(s, a)$

where W, D, L are the mean value estimates for the edge, $P(s, a)$ is the prior probability, and N is the visit count.

3.2.7 Exploration Mechanisms

To ensure robust exploration of the state space:

1. **Dirichlet Noise**: Applied exclusively to the root node prior probabilities: $P(s, a) = (1 - \varepsilon)P(s, a) + \varepsilon\eta$, where $\eta \sim \text{Dir}(\alpha)$ with $\alpha = 0.3$ and $\varepsilon = 0.25$.
2. **Temperature Scaling**: Move selection at the node is sampled proportionally to $N(s, a)^{\frac{1}{t}}$. The temperature t scales dynamically based on ply count and remaining material, transitioning from exploratory in the opening to deterministic in endgames.

3.2.8 Simulation Integrity and Masking

During MCTS rollouts, invalid state transitions poison the value estimations of the tree. To maintain simulation integrity, action masking is applied during the search phase across all configurations. However, in the "Unmasked" experimental configurations, the network's raw logits are not masked during the calculation of the loss function, forcing the network to internalize the penalty for assigning probability mass to illegal moves.

3.3 Reinforcement Learning Pipeline

3.3.1 Self-Play and Replay Buffer

The system generates data via highly parallelized self-play. Trajectories are stored in a First-In-First-Out (FIFO) replay buffer with a capacity of 524,288 samples. To prevent infinite games, rollouts are forcibly terminated as draws after 400 plies, or if the root value estimation for a draw exceeds 0.95, scaled to 0.75 in the endgame.

3.4 Optimisation and Loss Function

The network is optimized using the AdamW optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.99$ and weight decay $= 10^{-4}$. Noam learning rate scheduler is used with a factor of 0.1 and 4000 warmup steps.

For a given state s, let π be the MCTS visit count distribution, p be the network's policy prediction, z be the simulated game outcome, and v be the network's value prediction. The loss function is defined as: $L = (1 + \lambda)\left(-\sum_a \pi(a|s) \log p(a|s)\right) + (1 - \lambda)\left(-\sum_i z_i \log v_i\right)$

This acts as a strict mathematical curriculum. When the network proposes many illegal moves $\lambda \approx 1$, the policy loss approaches 2 and the value loss approaches 0. The network is forced to learn piece kinematics before state evaluation. As the model internalizes piece kinematics, the weights balance to $1 : 1$. This approach to curriculum learning is unique to the unmasked configuration, as λ is dependent on illegal move weights (λ is always zero in masked configurations).

3.5 Experimental Design and Evaluation Metrics

3.5.1 The Configuration Matrix

To isolate the effects of environmental guardrails, four distinct models are trained from scratch using identical hyperparameters (256 batch size, 256 MCTS simulations per move). The configurations form a 2×2 matrix:

1. **Legal + Masked (Control)**: Standard AlphaZero paradigm.
2. **Legal + Unmasked**: Network must learn piece kinematics via loss penalties before strategy.
3. **Pseudo-Legal + Masked**: Network plays the K_{cap} variant, and must learn FIDE rules before strategy.
4. **Pseudo-Legal + Unmasked**: Network must learn piece kinematics and FIDE rules entirely from scratch.

3.5.2 Quantitative Metrics

The learning dynamics of each configuration are evaluated using the following metrics, logged continuously during training:

- **Average Illegal Probability**: The total probability mass assigned by the raw policy head to invalid actions, quantifying the network's internalization of the ruleset.
- **Average Game Length**: Measures in plies, indicating the transition from random walks to strategic, prolonged play.
- **Win/Draw Ratios**: Tracking the emergence of decisive outcomes versus stalemates or repetitions.
- **Value Loss Convergence**: Measures the accuracy of the network's state evaluation over time.

Chapter 4

Implementation Details

This chapter details the software architecture and engineering paradigms utilized to construct the self-play reinforcement learning system. The implementation bridges the theoretical methodology with a highly optimized, parallelized execution environment. The system comprises a custom bitboard-based chess engine, a bipartite Monte Carlo Tree Search (MCTS) utilizing flat arena allocators, and a shared-weight Transformer encoder implemented via the Burn deep learning framework.

4.1 Software Stack and Overview

The system is implemented entirely in Rust, leveraging the language's strict aliasing rules and ownership model to guarantee data-race-free parallelization. Neural network operations are executed via the Burn framework, which provides a backend-agnostic tensor computation graph (WGPU for portable execution, CUDA for hardware acceleration).

The self-play pipeline (Algorithm 1) is designed for maximum throughput via lock-free concurrency. At the initialization of each training iteration, B parallel game instances are spawned. Because each MCTS instance encapsulates its own memory arenas, the Rayon library's `par_iter_mut()` is utilized to distribute tree traversals, leaf expansions, and backpropagations across all available CPU cores without mutex contention. Synchronization only occurs at the neural network interface, where leaf nodes from all B trees are aggregated into a single contiguous batch for GPU inference.

Reinforcement Learning Loop

```
Require: Model weights θ, Replay Buffer D (capacity 2^19), Batch size B
Initialize B independent MCTS instances {T_1, ..., T_B}
while training do
    for step = 1 to steps_per_iteration do
        for i = 1 to simulations_per_move do
            parallel for each tree T in {T_1, ..., T_B} do
                Traverse T to find leaf node n
            end
            Synchronize threads
            Batch inputs from all leaves and execute forward pass(θ)
            parallel for each tree T do
                Expand leaf n using network outputs
                Backpropagate value estimates to root
            end
        end
        parallel for each tree T do
            Select action a ~ π(a|s) based on visit counts
            Store transition (s, π, z) in D
            Advance environment state
            if terminal state reached then reset T
        end
    end
    if |D| > min_samples then
        for epoch = 1 to gradient_steps do
            Sample mini-batch from D
            Compute composite loss L(θ)
            Update θ via AdamW
        end
    end
end
```

4.2 Chess Engine

The environment is a custom chess engine optimized specifically for MCTS rollouts. State representation is strictly packed, and move generation relies on compile-time precomputed masks to eliminate branching and minimize CPU cache misses.

4.2.1 Bitboards and Move Generation

The board state is encoded within the `ChessBoard` struct using a `[[Bitboard; 6]; 2]` array, representing the 6 piece types across 2 colors, alongside 3 aggregate occupancy bitboards (White, Black, All). This yields a core memory footprint of 120 bytes, fitting

comfortably within two standard 64-byte CPU cache lines. Hardware intrinsics, specifically `trailing_zeros` and `leading_zeros`, are utilized for $O(1)$ piece iteration and bit isolation.

To accelerate move generation, the engine employs a two-step deferred verification architecture:

1. **Pseudo-Legal Generation:** Moves are generated branchlessly using lookup tables initialized via `const fn`. Sliding piece attacks are resolved using directional masks (`ROOK_ATTACKS`, `BISHOP_ATTACKS`) and a precomputed `BETWEEN` mask table. The engine isolates the first blocking piece using Least Significant Bit (LSB) or Most Significant Bit (MSB) isolation, applying the `BETWEEN` mask to enumerate valid target squares. Moves are written to a stack-allocated `ArrayVec<ChessMove, 128>`, safely bounding the empirical maximum branching factor of chess and entirely eliminating heap allocation overhead.
2. **Strict Legality Verification:** Rather than implementing computationally expensive pin-detection logic during generation, legality is verified post-hoc. The `is_legal()` method clones the board state, applies the pseudo-legal move, and evaluates if the resulting state leaves the active King under attack via `is_square_attacked()`. This deferred approach significantly reduces branching complexity while maintaining high throughput.

4.2.2 Zobrist hashing:

State repetition detection is implemented via Zobrist hashing. A static table of 64-bit pseudo-random keys (`ZobristKeys`) is lazily initialized using Rust's `OnceLock`. The hash is computed in $O(P)$ time, where P is the number of active pieces, by XORing the keys corresponding to the current board state, castling rights, en-passant file, and side-to-move. While incremental $O(1)$ hashing is theoretically optimal for sequential play, the $O(P)$ approach was selected to eliminate complex state-tracking across independent MCTS nodes. Profiling indicates this computational cost is negligible relative to network inference.

4.3 Bipartite Monte Carlo Tree Search

Standard MCTS implementations represent a complete action as a single edge. To map the search topology to the network's two-pass autoregressive policy, the MCTS is structured as a bipartite graph alternating between `PieceSelect` and `PieceMove` nodes.

4.3.1 Arena Allocators

To avoid the memory fragmentation and pointer-chasing overhead associated with recursive tree structures, the MCTS utilizes flat, index-based arena allocators:

- `node_arena: Vec<MctsNode>`
- `edge_arena: Vec<MctsEdge>`
- `position_arena: Vec<ChessPosition>`

Nodes reference their children and associated board states via `usize` indices. A dedicated `position_arena` is implemented because consecutive `PieceSelect` and `PieceMove` nodes share identical underlying board states (differing only in the `selected_sq` parameter). This deduplication provides a crucial memory optimization.

Architectural Limitation: The index-based arena design exhibits suboptimal memory scaling during deep rollouts. Because indices must remain stable throughout the lifetime of a game, fully explored or abandoned subtrees cannot be easily garbage-collected or culled without invalidating the index space. Consequently, RAM consumption scales disproportionately compared to VRAM during large batch operations, establishing system memory as the primary bottleneck for batch size scaling. Garbage collection methods were considered, but rejected to stay within scope of the project.

4.3.2 Node Expansion, Promotions and Terminal Handling

Leaf expansion (Algorithm 2) transitions the state machine. Expanding a `PieceSelect` edge generates a `PieceMove` node without altering the underlying board state. Expanding a `PieceMove` edge applies the move, computes the new Zobrist hash, evaluates terminal states (including threefold repetition via path history), and generates a new `PieceSelect` node.

Promotions are handled natively by the bipartite structure, bypassing the need for a third output dimension in the neural network. During the expansion of a pawn moving to the 8th rank, the MCTS intercepts the spatial prior p and distributes it uniformly ($\frac{p}{4}$) across four discrete edges (Queen, Rook, Bishop, Knight). This enables the model to handle underpromotions through search topology rather than increased output dimensionality.

Bipartite Leaf Expansion

```
Require: Leaf edge e, Network prior probabilities P, Search path history H
Let N_parent be the parent node of e
if N_parent is of type PieceSelect then
    Create N_child of type PieceMove
    Set N_child.selected_square = e.target_square
    Link e to N_child
else if N_parent is of type PieceMove then
    s' ← Apply move(N_parent.selected_square, e.target_square)
    if Zobrist(s') appears ≥ 2 times in H then
        Create N_child as Terminal(Draw)
    else if is_checkmate(s') then
        Create N_child as Terminal(Win/Loss)
    else
        Create N_child of type PieceSelect representing s'
    end
    Link e to N_child
end
if N_child is not Terminal then
    for each legal destination square i do
        if move is pawn promotion then
            Create 4 edges (Q, R, B, N) with prior = P[i]/4
        else
            Create 1 edge with prior = P[i]
        end
        Attach edge(s) to N_child
    end
end
```

4.3.3 Edge Selection and Value Propagation

During traversal, edge selection utilizes a PUCT variant that penalizes draws to encourage decisive variations. For a node with visit count N, an edge e with confidence p_e, visits n_e, and mean value (W_e, D_e, L_e) obtains a score edge_selection

- $Q_e = W_e - L_e - D_e$
- $U_e = p_e \cdot c_{\text{puct}} \cdot \frac{\sqrt{N}+10^{-8}}{1+n_e}$

The selected edge maximizes $Q_e + U_e$. Upon leaf evaluation, the value vector $[W, D, L]$ is backpropagated. If the side-to-move alternates along the path, the vector is inverted ($[W, D, L] \to [L, D, W]$) value_prop.

The path is cleared after each simulation, ready for the next traversal.

4.3.4 Mask Integration and Prior Renormalisation

In unmasked configurations, the network outputs a probability distribution over all 64 squares, including illegal moves. To maintain a valid probability distribution for PUCT, the probability mass assigned to illegal squares (λ) is calculated, and the legal priors are renormalized:

$$p'_i = \frac{p_i}{1-\lambda} \quad \forall i \in \text{legal moves}$$

In masked configurations, illegal logits are masked to -10^9 prior to the softmax operation, rendering renormalization unnecessary.

The illegal probability mass λ also plays a central role in the self-annealing loss described in Section 4.5.2.

4.4 Neural Network Architecture

The `ChessTransformer` is a shared-weight encoder mapping a spatial tensor and a metadata vector to a policy distribution and a value estimate.

4.4.1 Tensor Formulation and Canonicalisation:

To enforce symmetric learning, all board states are canonicalized to the perspective of the side-to-move via `flip_board()`. The input comprises:

- **Spatial Tensor ($\boldsymbol{X} \in \mathbb{R}^{64\times 14}$):** 12 piece planes, 1 en-passant plane, and 1 selected-square plane. The selected-square plane is populated exclusively during the `PieceMove` pass.
- **Metadata Tensor ($\boldsymbol{M} \in \mathbb{R}^{5}$):** 4 binary castling rights and 1 continuous scalar representing the half-move clock.

Tensors are constructed by extending flat `Vec<f32>` slices and reshaping them via Burn's `TensorData` API, ensuring contiguous memory layouts before transfer to the GPU.

4.4.2 Transformer Encoder and Dynamic Masking

The architecture utilizes $d_{\text{model}} = 512$, $n_{\text{layers}} = 8$, and $n_{\text{heads}} = 8$. The 64 spatial vectors are linearly projected to d_{model}, and 2D learnable positional embeddings are broadcast across the sequence. The metadata vector M is projected and concatenated as a pseudo-[CLS] token, resulting in a $65 \times d_{\text{model}}$ tensor.

Following the transformer layers, the sequence bifurcates:

- Tokens 0..63 are passed through a linear policy head to produce 64 spatial logits.
- Token 64 is passed through a linear value head to produce 3 logits (Win, Draw, Loss).

The model's `forward_classification` method dynamically applies action masking based on the experimental configuration. In masked configurations, a boolean tensor identifies illegal squares, and `mask_fill` is applied to set corresponding logits to -1×10^9 prior to softmax activation. In unmasked configurations, this step is bypassed, forcing the network to output raw logits over all squares.

4.5 Training Pipeline

The training pipeline orchestrates data generation, buffer management, and gradient updates, specifically engineered to support the study of rule internalisation.

4.5.1 Batch Expansion and Synchronisation

The interface between the CPU-bound MCTS and the GPU-bound neural network is implemented in `expand_batch` (Algorithm 3). During the self-play loop, MCTS traversals execute asynchronously across all threads until a leaf node is reached. The threads then synchronize, aggregating the canonicalized input tensors and legal move masks into a single batch. Following a unified forward pass, the resulting `NetworkLabels` (policy and value arrays) are scattered back to their respective MCTS instances for edge expansion and value backpropagation.

Batched Inference and Prior Renormalisation

Require: Set of leaf nodes $\{n_1, ..., n_B\}$, Configuration C

$X_{\text{batch}}, M_{\text{batch}} \leftarrow$ Extract spatial and meta tensors from $\{n_1, ..., n_B\}$
if C.masked is True **then**
  $M_{\text{batch}} \leftarrow$ Generate legal move boolean masks
else
  $M_{\text{batch}} \leftarrow$ None
end
$P_{\text{raw}}, V_{\text{raw}} \leftarrow \text{TransformerForward}(X_{\text{batch}}, M_{\text{batch}}, M_{\text{ask_batch}})$
for $b = 1$ **to** B **do**
  $\lambda \leftarrow 0$
  if C.masked is False **then**
    for each illegal square i in n_b **do**
      $\lambda \leftarrow \lambda + P_{\text{raw}[b]}[i]$
    end
    for each legal square j in n_b **do**
      $P_{\text{raw}[b]}[j] \leftarrow P_{\text{raw}[b]} \frac{[j]}{1-\lambda}$
    end
  end
  n_b.illegal_mass $\leftarrow \lambda$
  n_b.value $\leftarrow V_{\text{raw}[b]}$
  Populate n_b edges using $P_{\text{raw}[b]}$
end

4.5.2 Self-Annealing Loss Function

To facilitate learning in the unmasked configuration, a self-annealing loss function dynamically shifts the optimisation objective from kinematic legality to strategic evaluation. During batch expansion, the system calculates λ, representing the mean probability mass assigned to illegal moves across the batch.

Within the `calculate_loss` function, the composite loss is defined as:

$$L = L_{\text{policy}(1+\lambda)} + L_{\text{value}(1-\lambda)}$$

Where L_{policy} is the cross-entropy of the policy output against MCTS visit counts, and L_{value} is the cross-entropy of the value output against the terminal game outcome. When the network proposes a high volume of illegal moves ($\lambda \approx 1$), the value gradient is heavily suppressed. This acts as an automated, mathematically rigorous curriculum: the network is forced to optimize for piece kinematics ($L_{\text{policy}} \approx 2$) before strategic optimisation (L_{value}) can commence. As the model internalizes the rules ($\lambda \to 0$), the weights naturally balance to a $1:1$ ratio.

4.6 Performance and Correctness

Engine throughput is dictated by the interplay between CPU-bound tree search and GPU-bound tensor operations. Profiling indicates that the dominant computational cost is the batched forward pass of the Transformer.

The strict memory isolation of the MCTS arenas allows CPU scaling to remain perfectly linear until the GPU becomes saturated by the aggregated batch size. However, as noted in Section 4.3.1, the inability to cull unreferenced `ChessPosition` structs from the flat arenas results in high memory pressure. For a batch size of 256, concurrent games can consume upwards of 60 GB of system RAM during deep endgames, establishing memory capacity as the primary bottleneck for scaling the number of parallel self-play environments. Despite this, the architecture successfully sustains the high-throughput data generation required to train the four distinct agent configurations under identical hyperparameters.

Move generation is unit tested against multiple positions. Zobrist hashing uses 64 bit keys, providing cryptographic-level collision resistance for the search depth of the training configuration. Tensor shapes are checked at compile-time.

Chapter 5

Results and Evaluation

5.1 Experimental Setup

To ensure fairness, all variants were initialized with identical seeds, trained for the same number of epochs, and evaluated at identical MCTS search depths.

5.2 Ruleset Convergence Analysis

5.3 Masking and Grokking Analysis

Chapter 6

Conclusion

6.1 Project Evaluation

6.2 Limitations and Future Work

- **Compute Constraints:** Hardware limitations restricted the total number of training epochs, preventing observation of deep late-stage grokking.
- **Move Generation:** Future iterations should replace standard bitboard raycasting with Magic Bitboards for optimal

performance.

- **Endgame Tablebases (EGTB):** Injecting perfect knowledge at varying stages of the training pipeline remains a highly viable area for future research to bootstrap the value head and sharpen endgame play.
- **Engine Evaluations** Injecting perfect knowledge from high depth engine evaluations with forced wins similarly are viable for perject knowledge injection.
- **Kinematic Bootstrapping:** Boot strap the policy head with uniform distributions of valid moves, before playing for value.

Chapter 7

Statement of Ethics

Bibliography

Bengio, Y. *et al.* (2009) "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*. Montreal, Quebec, Canada: Association for Computing Machinery (ICML '09), pp. 41–48. Available at: https://doi.org/10.1145/1553374.1553380.

Huang, S. and Ontañón, S. (2020) "A Closer Look at Invalid Action Masking in Policy Gradient Algorithms," *CoRR* [Preprint]. Available at: https://arxiv.org/abs/2006.14171.

Jenner, E. *et al.* (2024) *Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*. Available at: https://arxiv.org/abs/2406.00877.

Monroe, D. and Chalmers, P.A. (2024) *Mastering Chess with a Transformer Model*. Available at: https://arxiv.org/abs/2409.12272.

Power, A. *et al.* (2022) "Grokking: Generalization beyond overfitting on small algorithmic datasets," *arXiv preprint arXiv:2201.02177* [Preprint].

Schrittwieser, J. *et al.* (2020) "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, 588(7839), pp. 604–609. Available at: https://doi.org/10.1038/s41586-020-03051-4.

Shannon, C.E. (1950) "XXII. Programming a computer for playing chess," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314), pp. 256–275. Available at: https://doi.org/10.1080/14786445008521796.

Sharma, S. *et al.* (2017) "Learning to Factor Policies and Action-Value Functions: Factored Action Space Representations for Deep Reinforcement learning," *CoRR* [Preprint]. Available at: http://arxiv.org/abs/1705.07269.

Silver, D. *et al.* (2017) *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*. Available at: https://arxiv.org/abs/1712.01815.

World Chess Federation (FIDE) (2023) "FIDE Laws of Chess". Available at: https://handbook.fide.com/chapter/E012023.